

Accountability Without Consensus—but Not Without a Witness: Verifiable Payment Provenance from Per-Party Signed Logs

bordumb · bordumbb@gmail.com

19 July 2026

Abstract

Autonomous software agents have begun to spend money, and the hard problem they raise is not payment but *accountability*: given a transaction an agent made, can anyone establish who authorized it, on what terms, and whether the agent stayed within them — without trusting the agent, its operator, or a central registry? We observe that this is the same problem a long line of *honesty machines* has attacked — medieval tally sticks, Renaissance double-entry bookkeeping (Pacioli 1494), cryptographically signed triple-entry receipts (Ijiri 1982; Grigg 2005), linked timestamping (Haber and Stornetta 1991), and finally the blockchain (Nakamoto 2008) — and that the last of these solves a strictly harder problem than accountability requires. Bitcoin needs global consensus because it prevents double-spending of a *bearer* asset, which forces every honest node onto one total order and thus into Byzantine agreement (Lamport, Shostak, and Pease 1982), with its attendant impossibility (Fischer, Lynch, and Paterson 1985) and replication cost. Accountability over *account-based* settlement carried by an external rail needs no such global order. We formalize the distinction and give a construction — self-certifying delegated mandates (Smith 2019; Dennis and Van Horn 1966) over per-party signed hash-chained spend logs, re-derivable offline by anyone — whose *tamper-evidence and authorization* reduce to the collision resistance of a hash and the unforgeability of a signature (Bernstein et al. 2012): no consensus, leader, or mining. We are deliberate about what this does not eliminate but *delegates*: single-settlement of the asset is handed to the external rail, and *freshness* — detecting a withheld or rolled-back log — still requires committing the head to an ordering **witness**, weaker than global consensus but not free. The construction adds nothing to the cryptographic toolbox; it *composes* established primitives — transparency-log tamper-evidence (Laurie, Langley, and Kasper 2013; Haber and Stornetta 1991), KERI delegated identifiers (Smith 2019), and attenuable capability tokens (Birgisson et al. 2014) — for the agent-payment setting, generalizing its nearest industrial precedent, a single-authority verifiable ledger (Amazon Web Services 2019), to the mutually-distrusting multi-party case. Verification is $O(1)$ per record and $O(n)$ storage per party. We prove tamper-evidence and authorization-unforgeability, show completeness against truncation is recoverable only under an anchor, and analyze the (borrowed) game theory of honest books against a measured receipt-lottery result (Naritomi 2019). The honest contribution is a *domain transfer plus composition*: carry KERI’s witnessed per-controller log from identity to payment provenance, add attenuable mandates and cost accounting, and reconcile pairwise through the rail — which adjudicates asset-uniqueness (the agreement-hard part) so the witness need only attest a single chain’s monotone growth, a strictly easier task. We keep Bitcoin’s tamper-evidence and, rather than *relocating* its consensus, *avoid* the agreement problem entirely — at the price of a witness for freshness and, candidly, no prevention of on-ledger collusion over rail-invisible metadata.

1. Introduction

1.1 The accountability gap

An autonomous agent that can spend money changes the interesting failure mode from a wrong answer to a wrong *transaction*: a loop that pays for the same call ten thousand times, a prompt-injected agent redirected to an attacker, an over-eager delegate that keeps buying until its cap is exhausted. The moment a principal hands spending power to a non-human agent, the first question anyone — the counterparty, an auditor, the principal themselves — must be able to answer is factual and prior to everything else: *did the agent act within the authority it was given?* Only after that question is settled can a dispute be resolved, a liability assigned, or a bad charge reversed.

Today that question has no good answer. The record of what an agent did is a log, and a log is only as trustworthy as the party that keeps it. What is needed is a record that is trustworthy *because of its structure* — one that any party can re-derive and check without trusting whoever produced it. This paper gives such a construction and, more importantly, argues that it needs far less machinery than the obvious answer (put it on a blockchain) would suggest.

1.2 Honesty machines

The problem is old; only the counterparties are new. For as long as there has been money there have been *honesty machines* — mechanisms that make two parties who might each prefer to lie commit to the same true record.

In medieval England the Exchequer settled debts with **tally sticks**: a notched stick split lengthwise, each party keeping a half that reconciled with the other only if neither was altered — forgery resistance from physical complementarity. In Renaissance Italy merchants formalized **double-entry bookkeeping** (Pacioli 1494): every transaction recorded twice, the books balancing only if both sides are honest, so fraud surfaces as an imbalance. Ijiri’s **triple-entry** proposal (Ijiri 1982) and, cryptographically, Grigg’s (Grigg 2005) add a third, shared, signed entry — the receipt both parties hold — so the record is not merely internally consistent but *mutually* attested. Haber and Stornetta’s **linked timestamping** (Haber and Stornetta 1991) chains hashes so that the order and content of a growing set of records cannot be changed without detection. And Bitcoin (Nakamoto 2008) combined the hash chain with a consensus rule so that a *network* of mutually distrusting nodes agrees on one ledger.

Each machine is the best its era’s technology allowed, and each pays a price: the tally stick needs both physical halves and a trusted master copy; double-entry needs an auditor and lives inside one firm’s books; the blockchain needs global agreement, and pays for it with mining, replication, and a liveness assumption. Our claim is that for *accountability* the blockchain’s price is not merely high but unnecessary.

1.3 Thesis: accountability is a local property

Bitcoin’s central achievement is preventing the double-spend of a *bearer* asset: a coin is the ledger entry, so two conflicting spends of the same coin must be globally excluded, which forces all honest nodes onto a single total order — i.e., Byzantine agreement (Lamport, Shostak, and Pease 1982), impossible in the asynchronous model with a single faulty process (Fischer, Lynch, and Paterson 1985) and, in practice, bought with proof-of-work.

Accountability asks a different question. When an agent pays a merchant, the money moves on an *external* rail (a card network, a stablecoin transfer, an escrow contract); the rail — not our system —

enforces that a dollar is spent once. What we must establish is *authorization and provenance*: that this settlement was within a signed mandate, in the correct amount, to a permitted counterparty, and that both parties' books and any third party reconcile to the same signed history. We show (§3) that this is a **local, per-party predicate** plus **pairwise reconciliation** plus **public re-derivability** — none of which requires agreement among the set of *all* parties, and therefore none of which requires consensus.

Two caveats belong up front, and §5 develops them. First, the load-bearing step is a *modeling* choice — delegating single-settlement to the rail — after which the formal result of §3 is nearly immediate; the interesting decision precedes the theorem. Second, tamper-evidence protects a log that is *presented*; detecting a *withheld* or rolled-back one — freshness — is the genuinely hard part, and there we reintroduce an ordering witness. So the precise claim is narrower than “consensus is unnecessary”: it is **tamper-evidence and authorization without consensus, freshness with a witness** — a real narrowing of the blockchain's job, not its wholesale elimination.

This is precisely the move KERI (Smith 2019) made for *identity*: replace a global blockchain of key events with per-controller **key event logs** that are self-certifying and witnessed, not consensus-ordered. We make the analogous move for *payment accountability*, and inherit KERI's identity layer as our root of trust. The construction (§4) is a delegated-capability mandate (Dennis and Van Horn 1966) over a per-party signed hash chain, verified offline; the security (§5) reduces to a hash and a signature, with none of consensus's assumptions; the economics (§6) are the game theory of honest books, honestly bounded. If there is a one-line version, it is a *domain transfer*: KERI carried identity off the global chain onto witnessed per-controller logs, and we carry payment provenance the same way — keeping Bitcoin's tamper-evidence while the **rail**, not a consensus, adjudicates the one globally-agreed fact (that a dollar is spent once). The catchy framing — Bitcoin's tamper-evidence without Bitcoin's blockchain — is true only with the asterisks §5 spends its length earning.

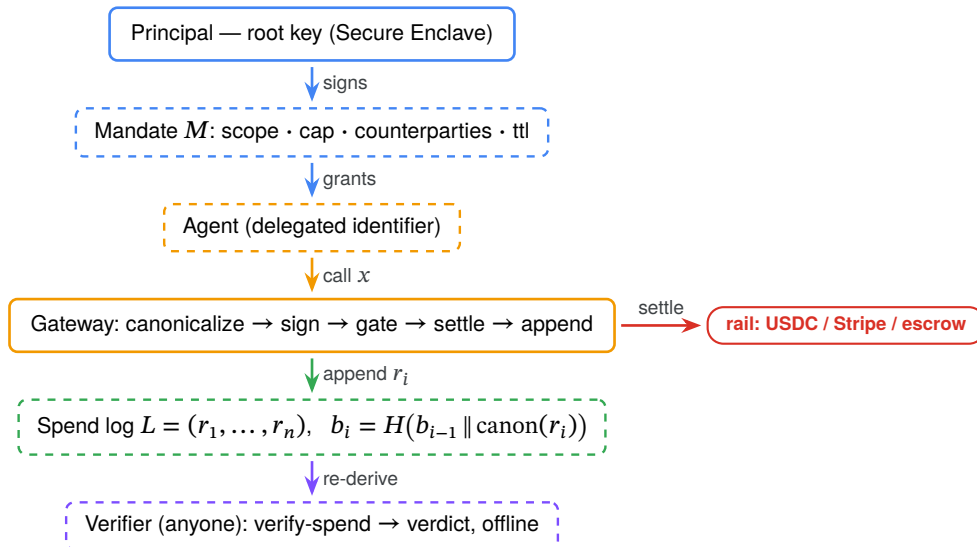


Figure 1: The construction. A principal's root key signs a scoped *mandate* to an agent; a gateway meters each call, settles it on an external rail, and appends a signed record to a per-party hash chain; anyone re-derives the chain offline to recover the verdict for every call. Nothing here is globally ordered or consensus-replicated.

2. Background and related work

Hash chains and timestamping. A cryptographic hash chain — $b_i = H(b_{i-1} \| m_i)$ — makes a sequence tamper-evident: altering any m_i changes b_i and every subsequent link. Lamport used a hash chain for one-time password authentication (Lamport 1981); Merkle’s hash trees (Merkle 1988) generalize the idea to efficient membership proofs; Haber and Stornetta (Haber and Stornetta 1991) built practical linked timestamping on it, the direct ancestor of the block header chain in (Nakamoto 2008). We use a per-party hash chain exactly as they did — for tamper-evidence — and pointedly *not* for consensus.

Transparency logs and verifiable ledgers. Making a hash-chained log *auditable by anyone* is the Certificate Transparency design (Laurie, Langley, and Kasper 2013) and its descendants (Google’s Trillian, and the Rekor log in Sigstore (Newman, Meyers, and Torres-Arias 2022)); a per-party, cryptographically verifiable, hash-chained ledger with *no consensus* is, almost exactly, Amazon’s QLDB (Amazon Web Services 2019) — the closest industrial precedent to this work.¹ The essential difference is trust topology: QLDB and CT logs assume a single trusted operator (the database owner, the log), whereas we target *mutually-distrusting* parties, so the log is per-party, the verdict is re-derived rather than asserted, and reconciliation is pairwise. We reuse their tamper-evidence wholesale and change only who must be trusted.

Consensus and its cost. Nakamoto consensus (Nakamoto 2008) solves Byzantine agreement (Lamport, Shostak, and Pease 1982) over an open membership set by making history expensive to rewrite. The FLP impossibility (Fischer, Lynch, and Paterson 1985) shows deterministic consensus is unattainable under asynchrony with even one crash fault, which is *why* real systems pay for it (proof-of-work, or a synchrony assumption, or a quorum). Our contribution is to show that the property needed for tamper-evidence and authorization is not agreement, so that tax does not apply there — while being candid (§5.3) that *freshness* still buys a lighter version of the same thing.

Self-certifying identity. KERI (Smith 2019) roots identity in self-certifying identifiers whose control is proven by replaying a per-controller key event log, witnessed rather than consensus-ordered. This is our identity substrate: an Autonomic Identifier is its own root of trust, and a *delegated* identifier lets a principal issue an agent a bounded, revocable sub-identity.

Capabilities and triple-entry accounting. The mandate is an object-capability (Dennis and Van Horn 1966): authority is a signed, unforgeable token conferring exactly a stated scope, not an identity on an access list — and specifically an *attenuable* one, in the lineage of macaroons (Birgisson et al. 2014) and biscuit (Couprie and the Biscuit authors 2021), whose caveats confine a bearer token’s scope offline and by anyone. The signed, mutually-held receipt is triple-entry accounting (Ijiri 1982; Grigg 2005) — the third entry that reconciles two parties’ books to one attested record. We add nothing to this toolbox; the contribution is the *composition* — attenuable capability delegation and triple-entry receipts over a per-party transparency log — aimed at agent payments.

Receipt lotteries. That honest bookkeeping can be induced by incentive rather than audit is not folklore: São Paulo’s consumer-reward program raised reported sales by at least 21% over four years, a rig-

¹Tellingly, Amazon *discontinued* QLDB entirely on 31 July 2025 (Amazon Web Services 2025), recommending migration to Aurora PostgreSQL — which drops the cryptographic verifiability, because the integrity guarantee lived in the data staying *where* and in the *form* it was created, so moving it elsewhere loses it. A verifiable ledger one can trust only while a single vendor keeps the service alive is exactly the failure this paper’s “re-derivable offline, trusting no vendor, including us” property exists to survive; the precedent’s death is itself an argument for the design.

orously measured effect (Naritomi 2019). §6 draws the analogy to Auths’s game theory precisely.

3. Why accountability needs no consensus

We make the thesis precise. Fix a hash H and a signature scheme Σ ; a **party** P keeps a spend log L_P (§4). Write the log’s **validity predicate** as the conjunction

$$\phi(L_P) \equiv \underbrace{\text{consistent}(L_P)}_{(a)} \wedge \underbrace{\bigwedge_{r \in L_P} \Sigma.\text{Verify}(r)}_{(b)} \wedge \underbrace{\bigwedge_{x \in L_P} (v(x) \text{ defined})}_{(c)},$$

where the underbraced conjuncts are (a) the hash-chain check — L_P re-derives to its committed head (Definition 1); (b) a valid signature on every record; and (c) a well-defined per-call verdict $v(x)$ (§4.3). Three observations separate this predicate from a blockchain’s requirement.

Locality. $\phi(L_P)$ is a predicate over a **single** party’s log. Its truth does not depend on any other party’s log, on a global clock, or on a shared ordering. It is checkable by reading L_P alone.

Pairwise reconciliation. When P pays Q , the shared fact is a single settlement s on the external rail (a transaction hash, a captured authorization). Agreement between P and Q means only that L_P and L_Q each reference s with the same amount and counterparty — a check over **two** logs and one external anchor, not over all parties.

Public re-derivability. Because ϕ is a function of L_P and public verification keys, **anyone** can recompute it. There is no privileged verifier whose word must be taken.

None of these is a global-agreement property. A blockchain is required precisely when the safety property is “*there is one history that everyone agrees on,*” because a bearer asset must not be spent twice anywhere. Here the rail already guarantees single-settlement of the asset, and Auths supplies only the orthogonal facts — *was it authorized, in what amount, by whom* — which are local. Formally:

Proposition 1 (No global order is required). Let $\mathcal{P}$ be any set of parties transacting pairwise over a rail that enforces single-settlement. Then the conjunction of (i) $\phi(L_P)$ for each P , (ii) pairwise reconciliation for each settled pair, and (iii) public re-derivability of both, is decidable by a verifier that reads only the logs and the rail, with no total order over $\bigcup_P L_P$ and no agreement protocol among $\mathcal{P}$.

Proof sketch. Each conjunct is a computable predicate over a bounded set of logs (one, or two, plus the rail record), evaluated independently; their conjunction over $\mathcal{P}$ is their pointwise evaluation. No conjunct references a variable shared across all of $\mathcal{P}$, so no cross-party synchronization or ordering is invoked. $\square$

We state plainly what is doing the work. Proposition 1 is nearly immediate — a conjunction of local predicates is local — and by itself proves little; the load-bearing step is the *modeling* decision that precedes it, delegating single-settlement to the rail. That decision is what makes provenance local, and it is also what excuses us from the double-spend problem — which is exactly the problem consensus exists to solve. So §3 is best read not as “we proved consensus unnecessary” but as “we chose a decomposition in which the hard part lives on the rail, and the remainder is local.” §5 examines the honesty of that trade.

The consequence is quantitative. Verifying a party's history is $O(n)$ in its own n records and $O(1)$ per record; there is no global state to replicate, no block interval, no mining. What Bitcoin spends on agreement, Auths simply never incurs, because it never asks the question agreement answers.

4. Construction

4.1 Identity and mandates

Following KERI (Smith 2019), every participant is an **autonomic identifier** (AID) I whose authoritative key state is derived by replaying its key event log K_I . A principal I_p delegates to an agent I_a by issuing an AID whose inception names I_p as delegator and is anchored in K_{I_p} ; control of I_a is provable, and revocable, by I_p alone.

A **mandate** (a delegated capability (Dennis and Van Horn 1966)) is a tuple

$$M = (I_p \rightarrow I_a, S, c, C, [t_0, t_1]),$$

with $I_p \rightarrow I_a$ the delegation from principal to agent, S a **scope** (permitted capabilities), c a **spending cap**, C a set of **permitted counterparties** ($C = \star$ meaning “any”), and $[t_0, t_1]$ a **validity window**. The tuple is signed under the principal's current key state and anchored in K_{I_p} , so the agent cannot widen it — only I_p 's key can, and doing so is itself a signed event. Here C is the machine-checkable form of “whom the agent may pay,” which we will see defeats the redirect attack.

4.2 The spend log as a per-party hash chain

Each metered call produces a record. A **call** is the tuple

$$x = (\kappa(x), \text{cost}(x), \text{cp}(x), \text{ts}(x), h(x)),$$

with $\kappa(x)$ the capability it exercises, $\text{cost}(x)$ its cost, $\text{cp}(x)$ its resolved counterparty, $\text{ts}(x)$ a timestamp, and $h(x)$ a hash of the (private) arguments. The gateway canonicalizes x into a record r_i , signs it under I_a 's device key, settles it on the rail, and appends it to the log $L = (r_1, \dots, r_n)$ with a running binding

$$b_0 = \text{seed}(I_a, M), \quad b_i = H(b_{i-1} \parallel \text{canon}(r_i)),$$

so the head b_n commits to the entire prefix. Each r_i carries b_{i-1} (its Auths-Prev) and I_a 's signature over $\text{canon}(r_i)$, including the cumulative $\sum_{j \leq i} \text{cost}(r_j)$ — a hash chain in the sense of (Haber and Stornetta 1991; Lamport 1981), per party, never merged into a global one. One wrinkle: the chain is linear, so concurrent calls by one agent must be **serialized** — a multi-instance gateway needs a single writer per agent to assign b_{i-1} (an atomic critical section). This caps a single agent's per-chain throughput, while distinct agents, on distinct chains, remain fully parallel.

Definition 1 (Consistent log). L is consistent if for every i , (a) $b_i = H(b_{i-1} \parallel \text{canon}(r_i))$, (b) $\Sigma.\text{Verify}$ accepts r_i 's signature under I_a 's key state as of $\text{ts}(r_i)$, and (c) the signed cumulative in r_i equals $\sum_{j \leq i} \text{cost}(r_j)$.

4.3 The per-call verdict

Authority is decided by a total function of proven facts. For a call x under mandate M and log prefix $L_{<x}$, define the **spent-before** quantity

$$\text{spent}(L_{<x}) = \sum_{r \in L_{<x}} \text{asrt}(r),$$

where $\text{asrt}(r)$ is the settlement amount the gateway *committed* into r at append time — a signed claim carried by the record, not a later ground-truth settlement. Then the verdict is

$$v(x) = \begin{cases} \text{unauthorized} & \text{if } I_a \text{ does not chain to } I_p, \text{ or a revocation } \leq b_n \text{ exists,} \\ \text{expired} & \text{else if } \text{ts}(x) \notin [t_0, t_1], \\ \text{out-of-scope} & \text{else if } \kappa(x) \notin S, \\ \text{out-of-counterparty} & \text{else if } \text{cp}(x) \notin C, \\ \text{over-budget} & \text{else if } \text{spent}(L_{<x}) + \text{cost}(x) > c, \\ \text{authorized} & \text{otherwise.} \end{cases}$$

The clauses are ordered — the first whose condition holds is the verdict — so each presumes the ones above it. An authorized call stayed within every dimension of the remit; every other verdict names *exactly how* it exceeded it. This predicate is the formal content of the question “did the agent exceed its authority?”

A subtlety the math hides but deployment does not: settlement on an external rail is *asynchronous* (authorize vs. capture, unconfirmed vs. confirmed), so the live gate and an offline verifier months later observe different *ground-truth* settlement states. Defining the verdict over the committed assertion $\text{asrt}(r)$ — a value fixed and signed into the record at append, not read from the rail at verification time — makes $v(x)$ a *total function of the signed prefix*, hence identical live and offline by construction (both re-run one implementation on one set of inputs). Whether each asserted amount actually settled is a separate question, resolved by *reconciliation* against the rail (Definition 2, §4.5) and surfaced as its own check: an assertion the rail never confirmed is flagged there and never silently rewrites a past verdict.

4.4 Re-derivation

The auditor `verify-spend` takes a log L (and the public K_{I_p}, K_{I_a}, M) and recomputes $b_1, \dots, b_n$, checks Definition 1, replays the KELs to establish key state and any revocation, and evaluates $v(x)$ for each call. It emits either `inconsistent` (naming the first failing check) or `consistent` together with the re-derived total and the vector of verdicts. An **evidence bundle** packages M , the relevant records, the settlements, and the verdicts, anchored *as of* head b_n (§5.3); a recipient with only the bundle and an open-source verifier re-derives the same conclusion with no network and no trust in the producer.

4.5 Cross-party reconciliation (triple-entry)

When P pays Q , both gateways meter the call, and — this is the load-bearing joint, so we make it explicit — each **binds the same settlement** into its record: r_i commits to $H(\text{rail}(s))$, the hash of the rail’s *authenticated* settlement record for s (its transaction identifier, amount, and, where the rail exposes it, the payee), which the rail either signs or itself publishes on a public ledger. This third, shared, rail-anchored entry is triple-entry accounting (Ijiri 1982; Grigg 2005).

Definition 2 (Reconciled pair). L_P and L_Q are *reconciled at s* if both are consistent (Definition 1), both contain a record committing to the same authenticated $H(\text{rail}(s))$, and the rail-adjudicated fields (amount, and payee where the rail pins it) agree with $\text{rail}(s)$.

Whether reconciliation is a strong guarantee or a weak one depends entirely on how much of the transaction the rail authenticates; Proposition 5 (§5.2) states exactly what it proves and, importantly, what it does not.

5. Security

5.1 Threat model

The adversary may control an agent (including a prompt-injected one), may operate its own gateway, and may attempt to fabricate, alter, reorder, truncate, or hide records; it may collude with a counterparty. It cannot forge Σ (EUF-CMA) or find H -collisions, and cannot produce key events under a key it does not hold. The rail is trusted only for single-settlement of the asset, not for authorization.

5.2 Tamper-evidence and unforgeability

Proposition 2 (Tamper-evidence). If H is collision-resistant, then any modification, insertion, deletion, or reordering of a consistent log L that preserves the head b_n yields an H -collision.

Proof. b_n is an iterated hash committing to the whole sequence, so a distinct log $L' \neq L$ with the same head is a second preimage of the chain. Since $b_n = b'_n$ but the sequences differ, let j be the largest index with $b_j = b'_j$ yet $(b_{j-1}, r_j) \neq (b'_{j-1}, r'_j)$; then $b_{j-1} \parallel \text{canon}(r_j)$ and $b'_{j-1} \parallel \text{canon}(r'_j)$ are distinct inputs with the same hash b_j — an H -collision (truncation is the special case where one side has no r_j). $\square$

Proposition 3 (Unforgeable authorization). Under EUF-CMA security of Σ and collision resistance of H , no probabilistic polynomial-time adversary lacking I_a 's key can produce a log that verify-spend accepts as consistent and in which some call x receives verdict authorized while x violates M (i.e., $\kappa(x) \notin S$, $\text{cp}(x) \notin C$, $\text{ts}(x) \notin [t_0, t_1]$, or the cap is exceeded).

Proof sketch. Consistency requires a valid Σ signature by I_a over each $\text{canon}(r_i)$; producing one without the key breaks EUF-CMA. Given valid signatures, $v(x)$ is a deterministic function of the signed fields and the KEL-derived key state, so an authorized verdict on a violating x requires either a signed record whose fields differ from x (again a forgery, or an H -collision to reuse a binding) or a KEL replay admitting I_a under I_p that I_p never anchored (a forged delegation event). Each reduces to the stated assumptions. $\square$

Two corollaries. **The redirect attack fails:** a prompt-injected agent that is in-scope and under budget but steered to a counterparty $\text{cp}(x) \notin C$ receives out-of-counterparty, refused live and re-derived identically offline — the mandate's C is what closes it, and because C is signed into M , an operator cannot loosen it without I_p 's signature. This holds only as strongly as revocation is *fresh*: the unauthorized clause tests for a revocation of the delegation, itself a freshness claim over the principal's KEL (§5.3), so a defense against a *just-revoked* key inherits the KEL-availability caveats below. **Collusion is disarmed only on rail-adjudicated facts.** Two parties who both wish to misreport the *amount* — or, on rails that pin it, the *payee address* — cannot: the rail record is authoritative and both chains commit to it (Definition 2), so a jointly consistent pair contradicting the rail is impossible without an H -collision. But the fields that carry the *accountability semantics* — the capability $\kappa(x)$, the counterparty label $\text{cp}(x)$

as distinct from a raw address, the purpose, and everything behind the private-argument hash $h(x)$ — are never seen by the rail. Two key-holders who agree to lie about those simply both sign the same lie: both chains re-derive as consistent, the settlement matches, and Proposition 3 does not touch them (it assumes a *single* adversary lacking a key). Triple-entry defeats *unilateral* restatement; it does nothing against *bilateral* agreement on what the money was for.

Proposition 5 (What reconciliation proves). Let L_P, L_Q be reconciled at s (Definition 2) with rail(s) authoritative. Then any disagreement between them on a *rail-adjudicated* field is **attributable**: at least one chain contradicts rail(s), and since the rail is authoritative that chain is the forger. On a *rail-invisible* field, a disagreement proves only that the chains *differ* — not which is correct — and agreement proves only that the parties told the *same* story, not a true one.

Cross-party accountability is therefore exactly as strong as the rail is expressive: the more of a transaction the rail authenticates (amount always; payee often; purpose almost never), the more of a dispute reconciliation can *adjudicate* rather than merely *record*. The honest summary is that collusion is *prevented* on rail-adjudicated facts and *tamper-evident but not prevented* on rail-invisible metadata — and, as §5.4 notes, colluders need not even go off-ledger; they can stay on-ledger and lie about purpose.

5.3 Completeness against truncation

This is where the “no consensus” claim acquires its asterisk, and we make it explicit. Propositions 2–3 defend a log that is *presented*; they do nothing against a party that **withholds** recent records or **rolls back** to an earlier head. Detecting that is a *freshness* problem, and freshness cannot be established from the log alone — it needs the head committed to some party trusted for **ordering**. Auths offers a ladder — a fleet treasury checkpoint, a witness/transparency co-signature (Smith 2019; Laurie, Langley, and Kasper 2013), an on-chain anchor, or, at the base, trust-on-first-seen — and the bundle **states which tier it used**. But note what every rung above the base *is*: a witness, a co-signer, or literally a chain — an ordering authority. So the precise, narrower claim is **tamper-evidence with no consensus; completeness only by reintroducing an ordering authority for freshness**. And that residual task is *strictly easier* than Bitcoin’s, in a way worth stating carefully. Bitcoin’s consensus solves a **uniqueness/agreement** problem: of two conflicting spends of one coin, all parties must agree that exactly one is valid, and agreeing on *which one wins under contention* is the part FLP (Fischer, Lynch, and Paterson 1985) makes expensive. We do not solve a cheaper version of that — we *avoid it entirely*. The rail adjudicates asset-uniqueness for us, so there is never a conflicting-spend tie that is ours to break; the witness need only certify that a *single* chain’s $(k, \text{cum}_k, \tau_k)$ grows monotonically — an **attestation**, not an agreement. “Freshness of one monotone log” is what remains, and it is a categorically smaller thing than consensus.

Proposition 4 (Anchored completeness). If a party publishes, at cadence, a signed anchor $a_k = \Sigma.\text{Sign}_W(\langle b_k, k, \text{cum}_k, \tau_k \rangle)$ by a witness W trusted for ordering, then any truncation below index k , or any rollback to an earlier head, is detectable: a later, valid anchor exists that a truncated log cannot reproduce, and any republished head with $\text{cum} < \text{cum}_k$ or $\tau < \tau_k$ is a W -signed contradiction.

Here a_k is a witness signature over $\langle b_k, k, \text{cum}_k, \tau_k \rangle$ — head, index, cumulative spend, and timestamp — published at cadence. The argument is monotonicity: an honest chain’s $(k, \text{cum}_k, \tau_k)$ only increases, so a verifier that has *witnessed* growth cannot be shown a smaller, “as-of-earlier” head without a signed contradiction (a ledger that jumps from ten thousand transactions a day to zero last week is not merely

suspicious — it is a signed self-contradiction). This lets a **privacy-preserving aggregate attestation** — a signed $\langle b_n, n, \text{cum}_n \rangle$, no per-record detail — prove ongoing activity without exposing the counterparty graph. The protection is exactly as real as the witness layer, and as the party’s actually anchoring.

5.4 Non-goals

Auths does not prevent off-ledger transactions (parties who never invoke it leave no record — the rail and the incentives of §6, not cryptography, pull activity onto it); it does not provide global double-spend prevention (delegated to the rail, by design); and it makes no **availability** guarantee. This last is the sharpest limit, and worth stating directly against the accountability goal: in a dispute the adversarial failure is rarely a *forged* log — tamper-evidence handles that — but a *missing* one. A party that simply declines to present its chain defeats the entire apparatus, and cryptography buys nothing there. The only real answer is, again, the witness/anchoring/pinning layer of §5.3 — the head co-committed, the records pinned by a third party — which is precisely where the trust removed at the identity and settlement layers returns at the availability layer. Put bluntly: the construction converts *forgery* (which it defeats) into *withholding* (which it does not, absent a witness), and the anchoring ladder is at once the least-developed part of the design and the most load-bearing for the accountability claim. Finally, it does not compose caps down a delegation tree: each mandate bounds its own chain, but aggregating several sub-delegations into a single principal-wide budget is itself a cross-chain reconciliation-and-freshness problem, out of scope here — which is why §1.1’s motivating failures are flat over-spend and redirect, not runaway sub-delegation.

6. The game theory of honest books

Why would a party keep its published history accurate when it could lie? The same reason a São Paulo merchant issues a receipt (Naritomi 2019): because the counterparty is enlisted against the fraud, and the uncertainty of who is checking makes continuous honesty the dominant strategy.

Cash tax evasion is a *collusion*: seller and buyer both gain by omitting the record. The receipt lottery changes the buyer’s payoff — the receipt is now a lottery ticket — so the buyer defects, and the merchant, not knowing which receipt will be audited, must issue them all. Naritomi measures the result: reported sales up at least 21% over four years, tax revenue net of rewards up 9.3% (Naritomi 2019). The mechanism is incentive alignment plus *uncertainty about who is watching*.

Auths reproduces both — with an honest asterisk. The counterparty is enlisted by construction (triple-entry, §4.5): a party who under-reports is contradicted by the other side’s chain and the shared settlement, and the hash chain makes a *presented* lie infeasible — not defected out of the collusion equilibrium but removed from it. But two disanalogies keep this from being self-enforcing. Nota Fiscal Paulista worked because the *state* ran the lottery, cross-checked filings, and handed the consumer a concrete positive payoff for demanding the receipt (Naritomi 2019) — the enforcement was real and central. Here, if publishing a log is optional and its availability is a non-goal (§5.4), “no party knows who will re-derive its log” loses its teeth: the deterrent is only as real as the witness layer, and that layer bites only if anchoring is real and somewhat mandatory — which again pulls back toward the trusted third party the construction wants to avoid. So the enforcement is *borrowed*: cryptography makes a presented lie infeasible; it is the witness/attestation layer, not the hash chain alone, that makes *not presenting* costly. With that layer real, one could even run the lottery itself over these receipts as a *provably fair* draw, trusting no authority; without it, the game theory is aspirational.

7. Discussion



Bitcoin. Prevents double-spend of a bearer asset — every node must agree on one chain.

Auths. Per-party signed hash chains + external settlement — no global order; anyone re-derives each chain.

Figure 2: The same hash-chain tamper-evidence, two different safety properties. Bitcoin globally orders a shared bearer ledger (consensus); Auths keeps independent per-party chains whose settlements reference an external rail (no consensus). Accountability lives entirely on the right.

The construction is deliberately conservative: it adds nothing to cryptography’s toolbox. Its contribution is a *reduction* — showing that the accountability problem raised by autonomous payments is the local, verifiable-provenance problem that hash chains and signatures already solve, not the global-agreement problem that motivated the blockchain. A single-authority verifiable ledger (Amazon Web Services 2019) already made this reduction for the *trusted-operator* case — and its 2025 retirement (§2) is itself a reminder that a ledger you can verify only while one vendor keeps it running is not the guarantee it appeared to be; our step is to carry the reduction to *mutually-distrusting* parties, whose logs re-derive with no operator at all, paying for it only in the freshness layer of §5.3. Where a system genuinely needs a shared bearer asset with no external settlement authority, consensus remains necessary; Auths simply observes that agent payments are not that system. The identity layer we build on (Smith 2019) made the same argument for key management a half-decade earlier, and the accounting tradition we formalize (Pacioli 1494; Ijiri 1982; Grigg 2005) made it, without the mathematics, for five centuries.

The practical payoff is that verification is cheap and universal: $O(1)$ per record, offline, by anyone, with no node to run and no token to hold. The evidence bundle a chargeback desk, an auditor, or a counterparty needs is a file, not a query against a chain — re-derivable on an air-gapped machine, trusting no vendor, including us.

8. Conclusion

We have argued that much of the accountability an autonomous-payments world demands — who authorized this, on what terms, and did the agent stay within them — is a local, per-party, publicly re-derivable property, and so does not need global consensus. We formalized what is and isn’t local (Proposition 1, honestly qualified), gave a construction of signed hash-chained spend logs under attenuable mandates (§4), proved its tamper-evidence and authorization-unforgeability (Propositions 2–3), formalized what cross-party reconciliation does and does not prove (Definition 2, Proposition 5), and showed completeness against truncation is recoverable only under an ordering witness (Proposition 4). The honest shape of the result is a **domain transfer plus composition**: KERI’s witnessed per-controller log, carried from identity to payment provenance, with attenuable mandates, cost accounting, and rail-anchored pairwise reconciliation. What lets it drop consensus is not a cleverer freshness mechanism but the **rail**, which adjudicates the one uniqueness/agreement fact — a coin is spent once — so the witness need only attest a single chain’s monotone growth. The system keeps the property Bitcoin borrowed from Haber and Stornetta — a history you cannot rewrite unseen — and, rather than *relocating*

Bitcoin’s consensus to a cheaper venue, *avoids* the agreement problem outright; a witness returns only for the strictly smaller task of freshness, and making that layer real — the answer to withholding, and to on-ledger collusion over rail-invisible metadata — is the design’s central unfinished work. So, with the asterisks earned: Bitcoin’s tamper-evidence, without Bitcoin’s consensus — and a witness where Bitcoin had a chain.

References

- Amazon Web Services. 2019. “Amazon Quantum Ledger Database (QLDB).” Managed ledger database: an immutable, cryptographically verifiable, hash-chained journal under a single trusted authority. <https://aws.amazon.com/qldb/>.
- . 2025. “AWS Services No Longer Available.” Amazon QLDB fully discontinued 31 July 2025; recommended migration to Aurora PostgreSQL. https://docs.aws.amazon.com/general/latest/gr/full_shutdown_services.html.
- Bernstein, Daniel J., Niels Duif, Tanja Lange, Peter Schwabe, and Bo-Yin Yang. 2012. “High-Speed High-Security Signatures.” *Journal of Cryptographic Engineering* 2 (2): 77–89. <https://doi.org/10.1007/s13389-012-0027-1>.
- Birgisson, Arnar, Joe Gibbs Politz, Úlfar Erlingsson, Ankur Taly, Michael Vrabie, and Mark Lentczner. 2014. “Macaroons: Cookies with Contextual Caveats for Decentralized Authorization in the Cloud.” In *Proceedings of the 21st Network and Distributed System Security Symposium (NDSS)*.
- Couprie, Geoffroy, and the Biscuit authors. 2021. “Biscuit: A Bearer Token with Offline Attenuation and Decentralized Verification.” Specification. <https://www.biscuitsec.org/>.
- Dennis, Jack B., and Earl C. Van Horn. 1966. “Programming Semantics for Multiprogrammed Computations.” *Communications of the ACM* 9 (3): 143–55. <https://doi.org/10.1145/365230.365252>.
- Fischer, Michael J., Nancy A. Lynch, and Michael S. Paterson. 1985. “Impossibility of Distributed Consensus with One Faulty Process.” *Journal of the ACM* 32 (2): 374–82. <https://doi.org/10.1145/3149.214121>.
- Grigg, Ian. 2005. “Triple Entry Accounting.” Systemics, Inc. https://iang.org/papers/triple_entry.html.
- Haber, Stuart, and W. Scott Stornetta. 1991. “How to Time-Stamp a Digital Document.” *Journal of Cryptology* 3 (2): 99–111. <https://doi.org/10.1007/BF00196791>.
- Ijiri, Yuji. 1982. *Triple-Entry Bookkeeping and Income Momentum*. Studies in Accounting Research 18. American Accounting Association.
- Lamport, Leslie. 1981. “Password Authentication with Insecure Communication.” *Communications of the ACM* 24 (11): 770–72. <https://doi.org/10.1145/358790.358797>.
- Lamport, Leslie, Robert Shostak, and Marshall Pease. 1982. “The Byzantine Generals Problem.” *ACM Transactions on Programming Languages and Systems* 4 (3): 382–401. <https://doi.org/10.1145/357172.357176>.
- Laurie, Ben, Adam Langley, and Emilia Kasper. 2013. “Certificate Transparency.” RFC 6962. Internet Engineering Task Force. <https://doi.org/10.17487/RFC6962>.
- Merkle, Ralph C. 1988. “A Digital Signature Based on a Conventional Encryption Function.” In *Advances in Cryptology — CRYPTO ’87*, 293:369–78. Lecture Notes in Computer Science. https://doi.org/10.1007/3-540-48184-2_32.
- Nakamoto, Satoshi. 2008. “Bitcoin: A Peer-to-Peer Electronic Cash System.” <https://bitcoin.org/bitcoin.pdf>.
- Naritomi, Joana. 2019. “Consumers as Tax Auditors.” *American Economic Review* 109 (9): 3031–72. <https://doi.org/10.1257/aer.20160658>.

- Newman, Zachary, John Speed Meyers, and Santiago Torres-Arias. 2022. “Sigstore: Software Signing for Everybody.” In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. <https://doi.org/10.1145/3548606.3560596>.
- Pacioli, Luca. 1494. *Summa de Arithmetica, Geometria, Proportioni Et Proportionalità*. Paganino de’ Paganini, Venice.
- Smith, Samuel M. 2019. “Key Event Receipt Infrastructure (KERI).” <https://arxiv.org/abs/1907.02143>.